

Da PoC à produção: JourneyGraph MNO sobre a infraestrutura real do CDRVIEW

Como as features validadas na demonstração (personas, IPED, Vamping, Markov/Bayes, detecção CDR-a-CDR) se encaixam nos quatro tiers já existentes na Claro — e o que muda de fato entre um pipeline Python de bancada e um sistema online, incremental e sob LGPD.

ENQUADRAMENTO

A PoC validou o método. A produção reaproveita a infraestrutura.

Toda a matemática construída na demonstração — K-Means+XGBoost para persona, IPED por trajeto, Vamping, cadeia de Markov e Bayes ingênuo para causa-raiz, MAD e filtro Bayesiano recursivo (Kalman) para outlier, teste binomial exato e Gama exato para o motor de alarme — é a mesma matemática que rodaria em produção. O que muda inteiramente é **onde** e **como** ela roda: a PoC é um script Python de execução única sobre um CSV estático de 4,6 milhões de linhas sintéticas; a produção precisa rodar de forma contínua, distribuída e sob controle de acesso real, sobre volume de CDR nacional — muitas ordens de grandeza acima do que a PoC processou.

A estrutura CDRVIEW já existente na Claro (69 VMs, 4 tiers) não é um destino a construir do zero: é a base sobre a qual o JourneyGraph MNO se encaixa como uma **camada de análise adicional**, consumindo o que o Parser/Ingestor, a Persistência/Redução e a Alarmística já produzem — e devolvendo à camada REST/API os indicadores que a GUI de Análises hoje já sabe exportar como tensores 3D/4D.

INFRAESTRUTURA DE REFERÊNCIA

Os quatro tiers do CDRVIEW hoje

Parser / Ingestor 30 VMs 16 vCPU · 80 GB RAM sem SSD dedicado	Persistência / Redução 30 VMs 16 vCPU · 80 GB RAM 2 TB SSD NVMe	REST / API + GUI 2 VMs 16 vCPU · 80 GB RAM 1 TB SSD	Alarmística 7 VMs 16 vCPU · 80 GB RAM 1 TB SSD
---	---	---	--

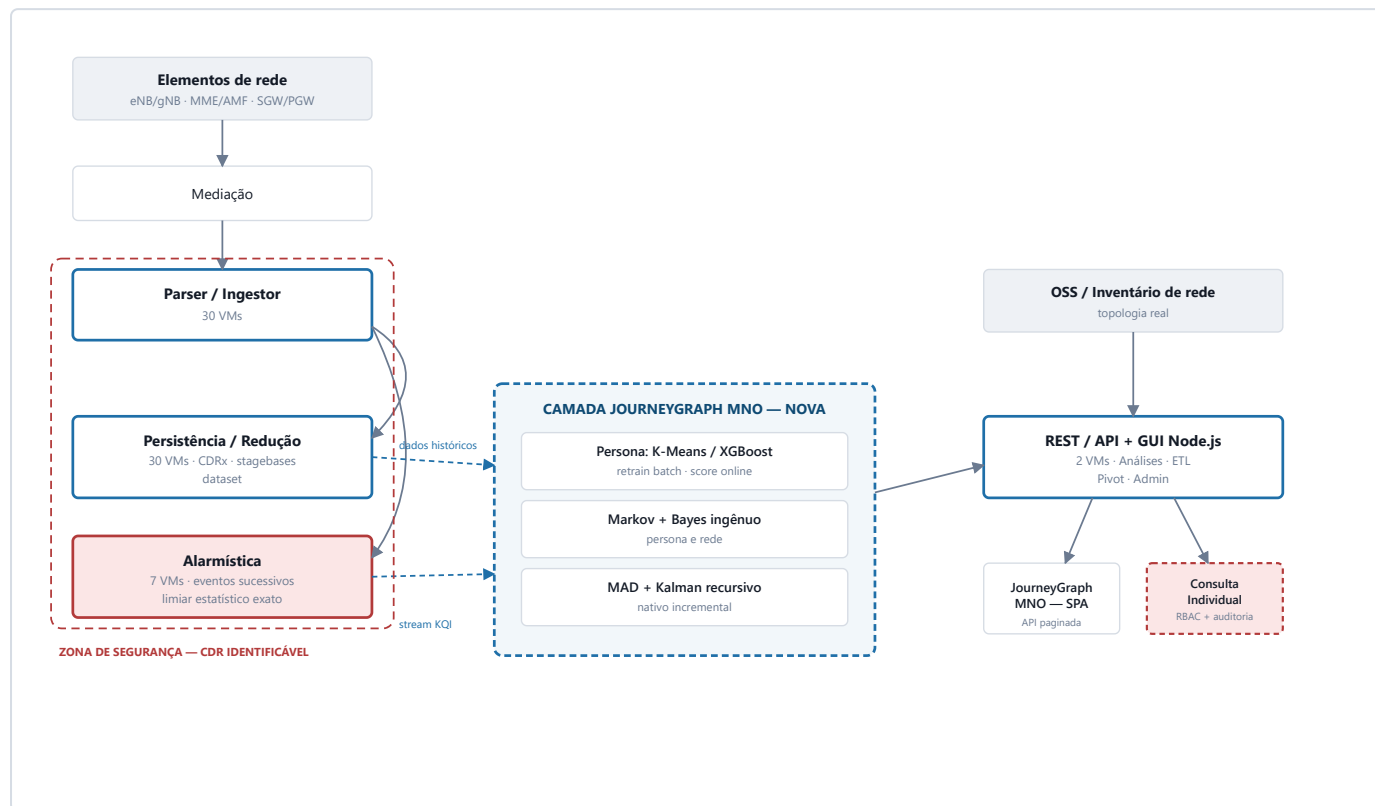
69 VMs · 1.104 vCPU · 5.520 GB RAM · ~69 TB SSD NVMe no total — dimensionado por particionamento horizontal (30+30 VMs sugerem sharding por região/comutador), não por instâncias verticalmente maiores.

Onde cada peça da PoC roda de verdade

COMPONENTE DA POC	TIER CDRVIEW	O QUE MUDA
00–01 · antenas + CDR sintético	n/a	Substituído pelo CDR real já recebido da mediação — sem geração sintética em produção.
02–03 · trajetos, tensores semânticos	Persistência/Redução	De pandas em processo único para job distribuído (Spark) lendo a base CDRx já particionada.
04–06 · features, K-Means, XGBoost	Persistência/Redução REST/API batch + inferência	Retreino periódico distribuído; <i>scoring</i> de persona incremental por assinante, não recomputo total.
14 · Vamping	Persistência/Redução	Agregação incremental diária (soma corrente por janela), não recomputo da janela inteira.
Topologia Core→PGW→SGW→Site	REST/API (GUI) + integração OSS/Inventário	Não inferir só do CDR — cruzar com o inventário real de rede (fonte de verdade), CDR só confirma presença/atividade.
Alertas Markov/Bayes (persona e rede)	Alarmística	Markov/Bayes ingênuo são adição nova — consumidores <i>read-only</i> do stream de KQI que a Alarmística já mantém.
Detecção CDR-a-CDR (eventos sucessivos + limiar exato)	Alarmística	já é a arquitetura real — a PoC replicou em lote o que a Alarmística já faz online, hoje.
Outliers (MAD + filtro Bayesiano recursivo)	Alarmística novo	O filtro de Kalman é nativo incremental por construção — encaixa direto no padrão de streaming já existente, sem adaptação.
App web (HTML único, JSON embutido)	REST/API + GUI (Node.js)	Vira cliente de API paginada. Nunca mais um artefato de 25 MB embutido — volume real inviabiliza esse padrão.

Fluxo de dados e novas responsabilidades

A camada nova (contorno azul) não substitui nenhum tier — ela consome a saída de dois deles (Persistência/Redução para histórico e tensores; Alarmística para o stream de KQI em tempo real) e devolve indicadores prontos para a GUI já existente na REST/API.



ZONA DE SEGURANÇA

Parser/Ingestor, Persistência/Redução e Alarmística tocam CDR identificável e devem ficar num segmento de rede isolado, sem exposição direta à camada de apresentação. A camada de Análises e o JourneyGraph MNO só devem enxergar dado já pseudonimizado/agregado — nunca o CDR bruto.

Onde e como implantar

- **Sharding consistente com o padrão existente:** os novos jobs de análise (persona, Markov/Bayes, agregações de IPED/Vamping) devem particionar por região/UF ou por comutador, espelhando o particionamento já usado no Parser/Ingestor e na Persistência/Redução — não introduzir um processo monolítico único onde os 60 VMs vizinhos já são distribuídos.
- **Separação Dev / Staging / Prod:** nenhum ambiente de desenvolvimento do JourneyGraph MNO toca CDR real de produção. Staging usa amostra sintética (o próprio gerador da PoC serve bem para isso) ou dado real anonimizado sob mascaramento irreversível.
- **Contêineres sobre a frota de VMs existente:** os novos jobs (Spark, serviços de *scoring*, o motor Markov/Bayes) podem ser containerizados e orquestrados dentro da mesma frota VM já provisionada, sem exigir uma plataforma de infraestrutura nova nem competir por VMs já alocadas ao CDRVIEW.
- **Alta disponibilidade por replicação de partição:** mesma lógica do resto do CDRVIEW — cada shard replicado em pelo menos 2 VMs, sem ponto único de falha na nova camada de análise.

LGPD como restrição de arquitetura, não checklist final

- **Pseudonimização desde a origem:** o padrão `assinante_hash` já usado na PoC deve ser real na produção — toda a camada de análise (persona, Markov/Bayes, dashboards) opera apenas sobre o hash, nunca sobre MSISDN. A reidentificação vive num serviço separado, estritamente controlado.
- **K-anonimato aplicado na borda da API,** não só no pipeline de geração — todo agregado exposto (trajeto, cluster, cela) precisa do mesmo piso $K \geq 3$ já usado na PoC, verificado no momento da resposta da API, não confiado ao consumidor.
- **RBAC por perfil de negócio:** Marketing/CRM enxerga só persona e jornada; NOC/Engenharia enxerga só rede (sem PII de assinante); Compliance/Jurídico é o único perfil com caminho para a Consulta Individual, sob as três hipóteses de uso já documentadas (mandado judicial, apuração de ouvidoria, SLA de NOC).
- **Auditoria imutável server-side:** o log de auditoria da PoC é ilustrativo e roda no cliente — inaceitável em produção. Produção exige log append-only, com identidade do operador, motivo declarado e timestamp, fora do alcance de qualquer edição pela própria aplicação.
- **Criptografia em repouso e em trânsito:** os 2 TB NVMe da Persistência/Redução criptografados em disco; mTLS entre todos os tiers, incluindo entre a nova camada de análise e a Alarmística/Persistência.
- **Texto normativo validado de verdade:** qualquer referência a RQUAL/Anatel exibida ao usuário final deixa de ser paráfrase ilustrativa e passa por validação da área regulatória antes de publicação.

O gargalo é volume, não o teste estatístico

Os testes exatos do motor de detecção (binomial, Beta, Gama) custam microssegundos cada — isso já foi medido na PoC: ~57 mil testes rodam em segundos num único processo Python. O que não escala num processo único é o **volume de CDR real**: a PoC processou 4,6 milhões de linhas sintéticas para uma cidade em 15 dias; o CDR nacional da Claro opera ordens de grandeza acima disso, continuamente — daí os 30+30 VMs já dedicados só a ingestão e persistência.

ETAPA	POC (BANCADA)	PRODUÇÃO (ALVO)
Motor de cálculo	pandas, processo único	Spark (ou equivalente), distribuído por partição
Retreino de persona	a cada execução manual	job agendado (ex.: semanal), versionado
Scoring de persona	recomputa a base inteira	incremental, por assinante, ao chegar CDR novo
Matriz de Markov	recontada da janela de 15 dias inteira	contador corrente + política de retenção/decaimento
Cache/materialização	nenhuma — HTML estático recalculado	views materializadas por período, servidas pela API

De execução única para stream contínuo

Esta é a mudança conceitual mais profunda entre PoC e produção. Cada componente tem uma resposta diferente para "como fica incremental":

COMPONENTE	NATUREZA INCREMENTAL	ESTADO
Filtro Bayesiano recursivo (Kalman)	já é nativo	Atualização O(1) por nova observação, por construção — nenhuma adaptação necessária, só plugar no stream real.
Eventos sucessivos	trivial	Contador de run corrente por recurso, incrementado ou zerado a cada evento — operador stateful simples em Flink/Kafka Streams.
Limiar estatístico (binomial/Gama exato)	já é a arquitetura real	Janela tumbling por contagem de evento — exatamente como a Alarmística já opera hoje.
Cadastro dinâmico de recurso	trivial	Contador corrente por recurso; vira "gerido" ao cruzar o limiar — sem recomputo.
Matriz de Markov (persona e rede)	requer desenho	Soma incremental de transições + política de janela/decaimento (ex.: exponencial ou N dias corrente) em vez de recontagem total.
IPED / Vamping	requer desenho	Agregação incremental por dia adicionado à janela corrente, não recomputo do período inteiro.
Persona (K-Means/XGBoost)	padrão MLOps	Retreino em batch periódico (o modelo muda pouco); inferência incremental por assinante a cada atualização de feature.
Topologia / centralidade / SPOF	evento de mudança	Estrutural — recomputa só quando o inventário de rede muda, não a cada CDR; o estado de saúde de cada nó (entrada do Markov/Bayes) é que é contínuo.

Limitações da PoC, sem eufemismo

A demonstração foi construída para validar **método**, não para ser um protótipo incremental do sistema final. Nada abaixo é surpresa — é o preço deliberado de rodar em bancada, uma cidade, dado sintético.

POC (HOJE)	PRODUÇÃO (EXIGÊNCIA REAL)
CDR 100% sintético e determinístico, uma cidade	CDR real, nacional, contínuo
Topologia de rede hash-atribuída (fictícia)	Topologia do inventário/OSS real, CDR só confirma atividade
Autenticação da Consulta Individual ilustrativa, client-side	SSO/LDAP, MFA, RBAC por papel, auditoria imutável server-side
Capacidade Engset é premissa (canais/setor não validado)	Capacidade real de engenharia RAN, por site
Execução única, recomputa tudo a cada rodada	Stream contínuo, estado incremental por recurso
App = 1 arquivo HTML com JSON de ~25 MB embutido	API paginada, cache/materialização por período
CRM/Ouvidoria sintéticos, gerados por seed	Integração real com sistemas de atendimento
Texto normativo (RQUAL/Anatel) é paráfrase para demo	Texto validado pela área regulatória antes de qualquer uso externo

Faseamento

● Fase 0 — Prova de método

CONCLUÍDA

Esta PoC. Validou que Markov, Bayes, MAD, Kalman, binomial/Gama exato e K-Means/XGBoost produzem resultado coerente e explicável sobre dado com estrutura realista.

● Fase 1 — Piloto com dado real, ainda em lote

1 REGIÃO REAL

Portar os scripts de persona/IPED/Vamping para rodar como job distribuído sobre a Persistência/ Redução, para uma região real, mantendo execução em lote (diária) — sem tocar a Alarmística ainda.

● Fase 2 — Integração com a Alarmística real

2 ALGORITMOS JÁ EXISTENTES

Conectar Markov/Bayes ingênuo e o filtro de Kalman como consumidores read-only do stream de KQI que a Alarmística já mantém — sem competir com o motor de eventos sucessivos/limiar exato que já roda lá.

● Fase 3 — Streaming completo + MLOps de persona

INCREMENTAL DE PONTA A PONTA

Scoring online de persona, Markov/IPED/Vamping incrementais, API paginada substituindo o artefato estático — JourneyGraph MNO deixa de ser um relatório e passa a ser um sistema vivo.

● Fase 4 — Rollout nacional

TODAS AS REGIÕES

Extensão do particionamento validado na Fase 1–3 para a cobertura nacional completa, com RBAC, auditoria e validação regulatória já maduros desde a Fase 2.